

Servo & Tracking II: Two-Degree-of-Freedom Control

Shaping the command response without touching the disturbance response.

A single feedback controller (1-DOF) has to do two jobs at once: reject disturbances and follow the reference. Tuning it aggressively for disturbance rejection often leaves an overshooting command response. A **two-degree-of-freedom** design adds a **prefilter** on the reference, decoupling the two jobs. In this tutorial you will:

- see a 1-DOF loop overshoot on a step command,
- add a reference prefilter that cancels the offending closed-loop zero, and
- confirm the disturbance response is unchanged (before/after).

Builds on PID Controllers/Intro.m. Run with `publish('TwoDOF.m')`.

Contents

- Plant and feedback controller
- The prefilter (the second degree of freedom)
- Command response: before vs. after
- What did NOT change: the disturbance response
- Try it yourself

Plant and feedback controller

Plant $G(s) = \frac{1}{s(s+2)}$ with a PI controller tuned for disturbance rejection. The PI zero ends up in the closed-loop transfer function and causes command overshoot.

```
G = tf(1,[1 2 0]);
Kp = 5; Ki = 3;
C = pid(Kp,Ki);           % (5s+3)/s, zero at s = -Ki/Kp = -0.6
T = feedback(C*G,1);      % reference -> output (1-DOF)
```

The prefilter (the second degree of freedom)

A unity-DC prefilter with a pole at the controller-zero location cancels that zero from the **command** path only.

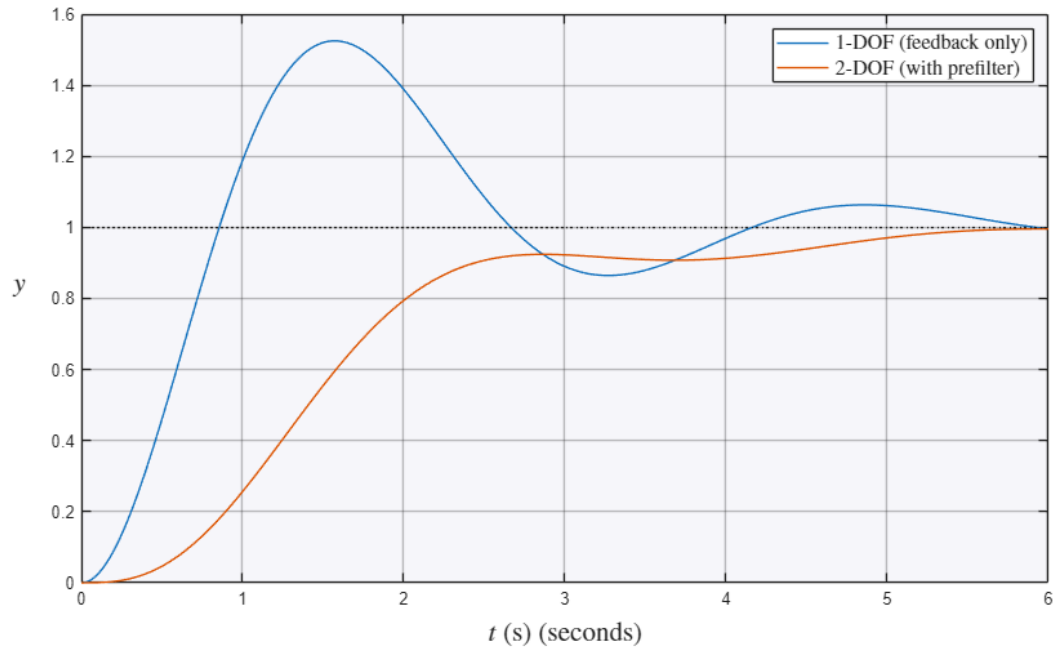
```
z = Ki/Kp;                % 0.6
F = tf(z,[1 z]);          % reference prefilter
T_2dof = F*T;              % reference -> output (2-DOF)
```

Command response: before vs. after

```
t = 0:0.01:6;
figure
step(T, t)
hold on
step(T_2dof, t)
yline(1,'k:', 'HandleVisibility','off')
hold off
grid on
legend('1-DOF (feedback only)', '2-DOF (with prefilter)', 'Interpreter','latex', 'FontSize',12)
title('Command Response: the Prefilter Removes the Overshoot', 'Interpreter','latex', 'FontSize',15)
ylabel('$y$', 'Interpreter','latex', 'FontSize',16); set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$ (s)', 'Interpreter','latex', 'FontSize',16)
fprintf('Command overshoot -- 1-DOF: %.1f%%, 2-DOF: %.1f%%\n', ...
    stepinfo(T).Overshoot, stepinfo(T_2dof).Overshoot)
```

Command overshoot -- 1-DOF: 52.4%, 2-DOF: 0.0%

Command Response: the Prefilter Removes the Overshoot

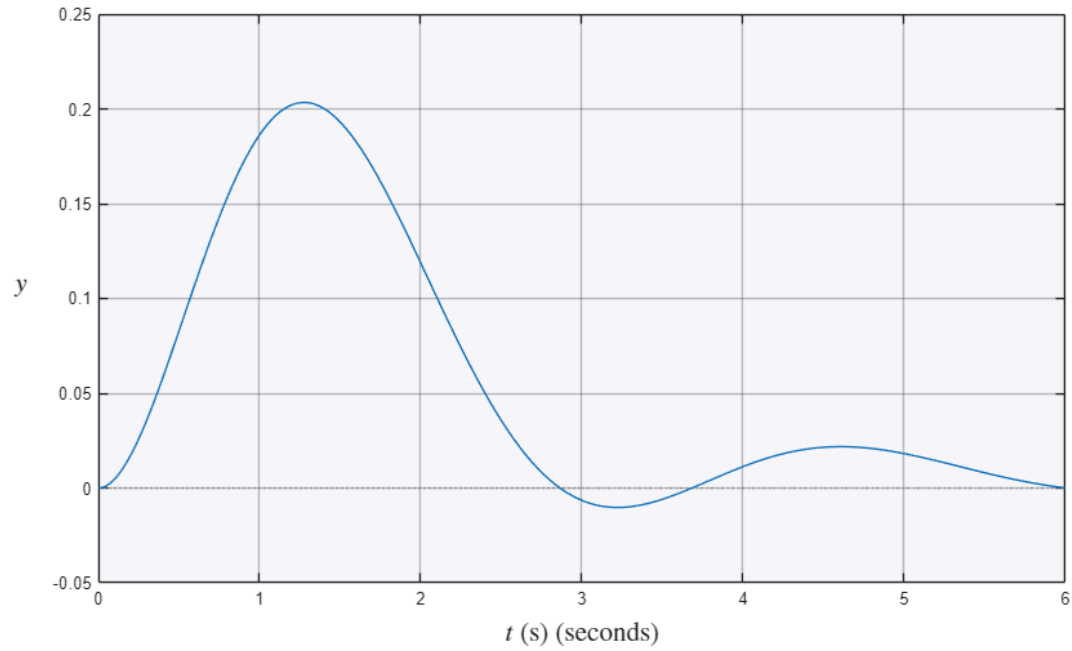


What did NOT change: the disturbance response

The prefilter only touches the reference, so disturbance rejection – the job the feedback was actually tuned for – is **identical** in both designs.

```
Td = feedback(G, C);           % input disturbance -> output
figure
step(Td, t)
grid on
title('Disturbance Response Is Unchanged by the Prefilter','Interpreter','latex','FontSize',15)
ylabel('$y$', 'Interpreter','latex','FontSize',16); set(gca,'YLabel','Rotation',0)
xlabel('$t$ (s)', 'Interpreter','latex','FontSize',16)
```

Disturbance Response Is Unchanged by the Prefilter



Try it yourself

- Detune the prefilter pole away from z and notice the overshoot creep back – the cancellation is what flattens the command response.
- Make the PI more aggressive (raise K_i) and watch the disturbance response improve while the prefilter keeps the command smooth.